

Self-Defeating Audits: Reversible Auditor-Blinding and Attribution Poisoning by a Low-Privilege Role in Trigger-Based Database Auditing

Thamilvendhan Munirathinam

ORCID: 0009-0005-5641-7424 DOI: 10.5281/zenodo.22004478

2026-08-19

Abstract

In-database audit trails are widely treated as ground truth for forensic reconstruction and compliance. We revisit that assumption for the most widely-copied trigger-based auditing pattern in PostgreSQL—an `AFTER` row trigger calling a `SECURITY DEFINER` function that serialises each write via an unqualified `row_to_json` into a privileged audit table—and ask a question the existing anti-forensics literature does not: *can the auditee itself, holding only the application’s privileges (as reached through SQL injection, with no superuser and no OS access), reversibly blind this auditor and poison attribution from within the database?* We build a disposable, synthetic-data lab and work a feasibility matrix cell-by-cell on stock PostgreSQL 14 and 16. Plain `search_path` shadowing of a catalog built-in fails (as expected, since `pg_catalog` is resolved first), but planting an *exact composite-type overload* of `row_to_json` in a writable `public` schema silently nulls captured rows against an unpinned `SECURITY DEFINER` trigger—reachable on stock PostgreSQL ≤ 14 , where `PUBLIC` holds `CREATE` on `public` by default, and misconfiguration-gated on 15+. Where the audit function is owned by the application’s own role, the attacker replaces its body to drop writes and then restores the original source byte-for-byte (function source is stored verbatim in `pg_proc`), leaving a clean object diff—while an independent write-versus-audit canary and a DDL event trigger still reveal the gap. We further demonstrate a `current_setting`-gated dormant bypass and, under a broad `INSERT` grant, fabricated back-dated audit rows indistinguishable from genuine ones using the audit table alone. The individual primitives are known—CVE-2018-1058 `search_path` shadowing and the `SECURITY DEFINER` escalation class—and we cite them as such; our contribution is their composition into a *self-defeating-audit* threat model at application privilege, an executed defense evaluation, and a reproducible artifact. We confirm the threat is a cross-engine class on the MySQL family (MariaDB 10.6/10.11) and SQL Server (2025/v17): the name-resolution shadowing class transfers to neither, but trigger-replacement, dormant bypass, and attribution poisoning do. MySQL’s `DEFINER`-privilege binding makes blinding more detectable, whereas SQL Server, like PostgreSQL, allows byte-perfect restore and additionally exposes a zero-residue `DISABLE TRIGGER` primitive its own DDL-trigger defense does not catch.

1 Introduction

Database audit trails underpin incident response, insider-threat investigation, and regulatory compliance. A pervasive folk design—reproduced across countless deployments from the PostgreSQL wiki “audit trigger” recipe—captures every write with a row-level trigger that serialises the changed row into an append-oriented audit table. Its appeal is that it lives *inside* the database, close to the data, and needs no external infrastructure.

The security literature on defeating such trails is almost entirely written from the perspective of a *privileged* adversary: a database administrator, a compromised superuser, or an insider who has escalated to administrative rights and simply disables logging or edits the log files [5]. That threat model is important but incomplete. The most common way an application database is actually attacked is SQL injection, which grants the attacker exactly the *application role’s* privileges—no more. This paper asks whether that low-privilege foothold is already sufficient to subvert the in-database auditor that is supposed to be recording the attack.

We answer empirically, with a reproducible lab, separating what is known from what is new. We contribute: (1) a **threat-model inversion**—reversible auditor-blinding and attribution poisoning by the *auditee* at application privilege against an *unaudited* auditor, a setting the privileged-adversary literature does not cover; (2) an **empirical feasibility matrix** on stock PostgreSQL 14 and 16, pinned to the exact version and **public-schema ACL** under which each result holds—the “structural on stock defaults” outcome survives only for PostgreSQL ≤ 14 ; (3) a **precise reversibility result**—because `pg_proc.prosrc` stores source verbatim, a naive semantically-equivalent restore is detectable but a capture-and-replay restore is byte-identical and defeats object-diffing; (4) two under-explored composition elements—a **current_setting**-gated dormant bypass and in-database attribution poisoning; (5) an **executed** defense $\times$ attack evaluation; and (6) a **cross-engine** generalisation to the MySQL family and SQL Server. We do not claim the underlying primitives as novel; they are CVE-2018-1058 and the `SECURITY DEFINER search_path` escalation class, and we position against them throughout.

2 Threat model

The adversary is the *application role*, reached via SQL injection into an application that connects as that role. It can issue `SELECT/INSERT/UPDATE/DELETE` on the business tables plus any DDL it is (mis)granted, and has: no superuser and cannot become one; no OS access (cannot touch log files, config, or the data directory); and no rights over the audit schema or its objects, except where a specific experiment models a common operator misconfiguration (stated explicitly). The defender is a DBA who set up trigger-based auditing following common guidance and inspects the database for tampering (object diffs, row counts) but does not necessarily run engine-level or out-of-engine logging. The security goal is audit *integrity* and *attribution*.

3 Background and related work

search_path and CVE-2018-1058. PostgreSQL resolves unqualified names against the schemas on `search_path`, with `pg_catalog` searched implicitly first unless placed explicitly. CVE-2018-1058 [1, 2, 3] established that “adding a schema to `search_path` effectively trusts all users having `CREATE` privilege on that schema.” Canonical mitigations are to schema-qualify calls, pin `search_path` (including to empty via `set_config`), and `REVOKE CREATE ON SCHEMA public FROM PUBLIC`—the last became the default ACL in PostgreSQL 15. Our vectors 1A/1B are instances of this class; we cite it as prior art and measure exactly when it does and does not yield audit-blinding.

SECURITY DEFINER functions. A `SECURITY DEFINER` function executes with its owner’s privileges. This is *necessary* for a trigger-based auditor: a low-privilege writer must cause an insert into a privileged audit table. The same feature is a documented escalation surface: an unpinned `SECURITY DEFINER` function resolving an unqualified name to an attacker-planted object runs it with the owner’s privileges [4, 2].

Anti-forensics assumes a privileged adversary. Prior work on database audit tampering and forensics consistently assumes an *administrative* adversary. The tamper-detection line [6, 7] designs hash-chained, externally-validated audit logs against an intruder who can silently corrupt stored records—i.e. someone with direct write access to DB internals. Storage-level breach detection [8] is explicitly motivated by “an attacker with administrator privileges who could otherwise cover their tracks by manipulating audit log files,” and blockchain-for-DBMS integrity [9] assumes “administrative privileges” able to “alter logs and login records.” MemTraceDB [5] likewise assumes an attacker “escalated... to an administrative level.” The practitioner baseline agrees: trigger auditing is held to be bypassable only by “superusers (or table owners)” who can disable triggers, and pgAudit “cannot reliably audit superusers” [11]. The one genuine prior art for a *low-privilege* role subverting trusted PostgreSQL machinery is CVE-2018-1058 [1]: any user who can create in a writable `public` can hijack a higher-privileged victim’s function resolution. We build directly on that primitive but target a different asset—the *audit trail* itself (reversible blinding, dormancy, attribution poisoning)—which that line does not consider. We are not aware of prior work evaluating reversible blinding of an unaudited trigger-based auditor by a low-privilege, injection-only role.

Tamper-evident logging. That an unaudited log is tamperable is textbook: a tamper-evident log “requires frequent auditing” by “at least one auditor... assumed to be incorruptible” [12]. Append-only storage is necessary but not sufficient without cryptographic tamper-evidence—forward integrity [10], forward-secure/hash-chained logs [13], or transparency-log signatures [14].

4 The victim

We build the representative victim as the DBA. A business schema `app` owned by the application role holds `app.accounts`. A separate audit schema owned by the DBA holds `audit.logged_actions`. The trigger function is:

```
CREATE FUNCTION audit.if_modified()
RETURNS trigger AS $$
DECLARE payload jsonb;
BEGIN
  IF (TG_OP='DELETE') THEN
    payload := row_to_json(OLD)::jsonb;
  ELSE
    payload := row_to_json(NEW)::jsonb;
  END IF;
  INSERT INTO audit.logged_actions(
    table_name,action,row_data)
  VALUES(TG_TABLE_NAME,left(TG_OP,1),payload);
  RETURN COALESCE(NEW,OLD);
END; $$ LANGUAGE plpgsql SECURITY DEFINER;
```

fired AFTER INSERT OR UPDATE OR DELETE ...FOR EACH ROW. The application role is granted only USAGE on `app` and DML on `app.accounts`. One faithful correction to the naive wiki snippet: the function *must* be SECURITY DEFINER, else a low-privilege writer’s own trigger-driven insert into the audit table fails with a permission error rather than auditing—and it is exactly this SECURITY DEFINER-with-mutable-search_path shape that is the CVE-2018-1058 surface.

5 Methodology

We run each cell in a disposable Docker container (`postgres:14-alpine`, `postgres:16-alpine`; image digests recorded) against a freshly built victim, and *normalise* three preconditions to a known state so results are comparable across versions: `pub_create` (does `PUBLIC` hold `CREATE` on `public`), `pin_path` (is the trigger’s `search_path` pinned), `app_owns` (does the app role own/`ALTER` the audit function). Separately we record the *true stock default* per version by probing a fresh database with a fresh low-privilege role. Blinding is measured *out of band*: each attack plants a uniquely-named sentinel write, and the DBA counts whether it reached the audit table—the ground truth of whether auditing occurred, independent of whether the attack’s setup statements succeeded. All identities and addresses are synthetic (RFC 5737 documentation ranges [15]).

6 Results

6.1 Feasibility matrix (RQ1)

Behaviour is identical on PostgreSQL 14 and 16 at the mechanism level; the difference is in what counts as “stock.”

Vector	Blinds?	Precondition it truly needs
1A plant shadow in a <i>new</i> schema	No (setup-blocked)	fails at <code>CREATE SCHEMA</code> (app lacks <code>CREATE-on-db</code>) — a privilege barrier, not name resolution
1S same-signature, <code>public</code> first	Yes (payload-null)	writable <code>public</code> ; plain CVE-2018-1058 schema ordering
1T exact overload, <code>pg_catalog</code> first	Yes (payload-null)	writable <code>public</code> ; isolates type-specificity (exact match beats built-in despite catalog first)
1C replace trigger fn body	Yes (no record)	app owns / can <code>ALTER</code> the audit fn
1D bespoke secdef helper	N/A	none beyond the built-ins above

The decisive, verified version nuance: on stock PostgreSQL 14 a fresh low-privilege role *can* `CREATE` in `public` (ACL grants `PUBLIC USAGE+CREATE`), so the overload shadow is reachable on defaults; on stock 15+/16 `PUBLIC` has only `USAGE`, so it needs an explicit re-grant or another writable schema on the function’s path. We stress a correction over an earlier draft: 1A does *not* show that “`pg_catalog` always wins”—that would contradict CVE-2018-1058. 1A fails purely because a fresh role cannot create a schema; when the attacker plants into the *already-writable* `public`, shadowing succeeds (1S, by schema ordering), and the exact-rowtype overload wins *even with* `pg_catalog explicitly first` (1T), isolating best-type-match from schema order. Note the distinction: the overload vectors (1S/1T) null the captured payload while still writing an attributed, timestamped audit row (*payload-nulling*); only 1C removes the record entirely.

6.2 Reversibility (RQ2)

In 1C the attacker captures the exact original definition (`pg_proc.prosrc` is world-readable and stored *verbatim*), replaces the body with a no-op to drop one write, then replays the captured definition byte-for-byte: `pg_get_functiondef` is identical before and after, so a source/object diff finds nothing. This is not a tautology but a contrast: our negative control (1R2) restores a *behaviourally-identical but reformatted* body and the function md5 changes (`66cf56bc...` → `bab9fc81...`), so a naive restore *is* detected—only the verbatim capture-and-replay defeats the diff. What still betrays even a perfect restore is *independent* of the audit code: a write-versus-audit

Table 1: Cross-engine comparison of the self-defeating-audit threat.

Aspect	PostgreSQL (14/16)	MySQL family (MariaDB)	SQL Server (2025)
“Definer” mechanism	SECURITY DEFINER fn	DEFINER trigger	ownership chaining
Shadow class (1A/1B)	blinds on stock ≤ 14	N/A (no search_path)	N/A (no search_path)
Replacement precondition	owns the audit fn	TRIGGER priv on table	ALTER on table
Replacement (1C)	blinds; byte-perfect restore	blinds; cannot restore DEFINER (residue)	blinds; byte-perfect restore
Extra blinding primitive	—	—	DISABLE TRIGGER (zero residue)
Dormant bypass	easy	harder (needs audit-INSERT)	easy
Attribution poisoning	indistinguishable	identical	identical
Engine-level defense	pgaudit/log_statement	general_log /binlog	SQL Server Audit
Detection nuance	DDL event trigger	object-diff (missing trigger)	DDL trigger misses DISABLE

canary shows a gap of one, and a DDL event trigger recorded two **CREATE OR REPLACE FUNCTION** events. Object-diffing the audit code is therefore insufficient on its own.

6.3 Dormant bypass (RQ3) and attribution poisoning (RQ4)

Given the 1C ownership precondition, a **current_setting('app.maint')**-gated branch skips logging only in a session that sets the flag: the gated write is not logged while a control write in the same session is. Given a broad **INSERT** grant, the app role writes a fabricated row—non-existent user, synthetic documentation IP, back-dated three days—schema-identical to genuine rows; a reviewer using only the audit table has no field that flags it as forged. Exposure requires cross-layer correlation with an app-side log.

6.4 Cross-engine generalisation

We reproduced the study on the MySQL family (MariaDB 10.6/10.11) and SQL Server (LocalDB, 2025/v17). The composition transfers to all three; the surface and reversibility differ (Table 1). The shadowing class transfers to neither MySQL nor SQL Server (no **search_path**). On MySQL, whose **DEFINER**-trigger model is the analogue, trigger replacement needs the **TRIGGER** privilege, but the **DEFINER** binds privilege so the attacker cannot recreate the original **root-DEFINER** trigger without **SET USER/SUPER**—a missing/alterd trigger is an un-erasable residue, and dormancy additionally needs an audit-INSERT grant. On SQL Server the “definer” analogue is ownership chaining; replacement needs **ALTER** on the table and, because module source is stored verbatim, the attacker restores byte-for-byte (clean diff, like PostgreSQL); dormancy is easy (an **ALTER** keeps **dbo** ownership so chaining survives). SQL Server also exposes a zero-residue **DISABLE TRIGGER** primitive that raises no DDL event, so a DDL-trigger guard catching **ALTER** does not catch it.

7 Defenses (RQ5)

Every defense below was executed against the live attacks, not asserted.

Key executed results: pinning the trigger **search_path** turns 1B from blinded to logged even with **public** writable; correct ownership makes the app role unable to **CREATE OR REPLACE** the function, killing 1C and the dormant bypass; the DDL event trigger cannot be dropped by the app role and records the replacements; the hash-chained sink’s verifier returns the tampered sequence

Attack ↓ / Defense →	Pin path / REVOKE public	Ownership / priv	Engine log + hash-chain
1B overload shadow	stops	—	logs write
1C replace fn/trigger	—	stops	logs + detects
2 dormant	—	stops	logs + detects
3 attribution poison	—	—	exposes (chain break)

number on a raw/back-dated insert (NULL on an intact chain); and engine-level `log_statement` captures the write in the server log even while the trigger auditor is blinded—the app role can neither change the GUC (superuser-only) nor reach the log file. Only engine-level logging plus an out-of-engine tamper-evident sink closes the whole set. The same pattern holds cross-engine (`general_log/binlog` and SQL Server Audit; remove the `TRIGGER/ALTER` privilege; hash-chained sink).

8 Discussion

What is and is not novel. The primitives—`search_path` shadowing (CVE-2018-1058) and `SECURITY DEFINER` escalation—are prior art and we claim neither. The contribution is (i) the threat-model inversion to a low-privilege, injection-only auditee against an unaudited auditor; (ii) the composition of reversible blinding, dormant bypass, and in-database attribution poisoning; (iii) the verbatim-restore reversibility observation; (iv) an executed cross-engine defense evaluation and reproducible artifact.

Honest scoping and significance. The strongest “works on stock defaults” claim holds only for the overload-shadow class (1S/1T/1B) on PostgreSQL ≤ 14 ; on 15+ the `public`-schema default change makes it misconfiguration-gated. We state the version and `public`-schema ACL for every result. We are also explicit that 1C, dormancy, and attribution poisoning *presuppose an audit-object misconfiguration*: the app role already owns/can `ALTER` the audit function, or can `INSERT` into the audit table. Their contribution is not “tampering is possible” (expected once the auditee controls the auditor) but *what such a common misconfiguration additionally unlocks*—clean-object-diff reversibility, session-gated dormancy, and forensically-indistinguishable poisoning—and how detectably each manifests, which differs sharply across the three engines.

Limitations. We evaluate PostgreSQL (14, 16), the MySQL family (MariaDB 10.6, 10.11), and SQL Server (2025/v17); Oracle is future work. The SQL Server experiments run on LocalDB and model the low-privilege attacker via `EXECUTE AS USER` impersonation (LocalDB is Windows-auth only), which enforces that principal’s permissions but is not a separate network session. The victim is the wiki pattern; bespoke auditors may pin `search_path` or use engine logging already. We measure blinding of a single logical write, not throughput-scale evasion.

9 Responsible disclosure

This work was conducted entirely in disposable local labs with synthetic data and no real system identified. The behaviours shown are documented engine behaviour plus operator misconfiguration, not new engine defects; a courtesy note was sent to the PostgreSQL security team framed as such. The published SQL is scoped to the lab schema and is not a weaponised payload.

10 Reproducibility

The complete lab—container setup, parameterised victims, attack catalogues, defenses, snapshot tooling, and the orchestrators that fill the matrices—is released as an artifact with pinned image digests and per-version evidence logs, archived at Zenodo (DOI: 10.5281/zenodo.22004478) and mirrored on GitHub (https://github.com/mthamil107/DB_Audit_Research).

11 Conclusion

A SQL-injected application role, with no superuser and no OS access, can reversibly blind the widely-copied trigger-based auditor and poison attribution from within the database—under pre-conditions that are stock defaults on PostgreSQL ≤ 14 and common misconfigurations elsewhere. The individual mechanisms are known; their composition into a self-defeating audit at application privilege is not addressed by existing anti-forensics work, which assumes a privileged adversary. The remedy is unambiguous: do not trust an in-engine trigger log as tamper-proof against the role it audits—pin `search_path`, revoke `CREATE` on `public`, own audit objects with a role the application can never alter, and record at the engine into an out-of-engine, hash-chained, append-only sink.

References

- [1] PostgreSQL Global Development Group. *CVE-2018-1058*. <https://www.postgresql.org/support/security/CVE-2018-1058/>
- [2] PostgreSQL. Documentation commit on `search_path` trust (CVE-2018-1058 fix). <https://github.com/postgres/postgres/commit/5770172cb0c9df9e6ce27c507b449557e5b45124>
- [3] PostgreSQL Wiki. *A Guide to CVE-2018-1058: Protect Your Search Path*. https://wiki.postgresql.org/wiki/A_Guide_to_CVE-2018-1058
- [4] Cybertec. *Abusing SECURITY DEFINER functions*. <https://www.cybertec-postgresql.com/en/abusing-security-definer-functions/>
- [5] *MemTraceDB*. arXiv:2509.05891. <https://arxiv.org/pdf/2509.05891>
- [6] R. T. Snodgrass, S. S. Yao, C. Collberg. *Tamper Detection in Audit Logs*. VLDB 2004, pp. 504–515.
- [7] K. E. Pavlou, R. T. Snodgrass. *Forensic Analysis of Database Tampering*. ACM TODS 33(4), Art. 30, 2008. <https://doi.org/10.1145/1412331.1412342>
- [8] J. Wagner et al. *Carving Database Storage to Detect and Trace Security Breaches*. Digital Investigation 22 (DFRWS USA 2017), S127–S136. <https://doi.org/10.1016/j.diin.2017.06.006>
- [9] S. S. Srivastava et al. *Verity: Blockchains to Detect Insider Attacks in DBMS*. arXiv:1901.00228. <https://arxiv.org/abs/1901.00228>
- [10] M. Bellare, B. Yee. *Forward Integrity for Secure Audit Logs*. Tech. Report, UC San Diego, 1997.
- [11] pgAudit — PostgreSQL Audit Logging, and PostgreSQL Wiki, *Audit trigger*. <https://github.com/pgaudit/pgaudit>

- [12] S. A. Crosby and D. S. Wallach. *Efficient Data Structures for Tamper-Evident Logging*. USENIX Security 2009.
- [13] B. Schneier and J. Kelsey. *Cryptographic Support for Secure Logs on Untrusted Machines*. USENIX Security 1998.
- [14] B. Laurie, A. Langley, E. Kasper. *Certificate Transparency*. RFC 6962.
- [15] J. Arkko, M. Cotton, L. Vegoda. *IPv4 Address Blocks Reserved for Documentation*. RFC 5737.